

Phase 9.5 encryption — adversarial re-audit

Date: 2026-08-18 **Why:** the 2026-08-09 audit ran four passes producing 25 findings, but its verification pass died on a session limit after 7. Eighteen findings were never verified and **their text was never written down** — no journal survived, so they could not be recovered. This run re-derives them from scratch and records every one, verdict included, so the same loss cannot happen again.

Method: six independent attack lenses (crypto core, backfill, gate, scope coverage, operational/data-loss, test quality), each capped at six findings. Every finding was then handed to a separate skeptic agent instructed to REFUTE it by reading the code, defaulting to “refuted” unless it could trace the failure path itself.

Honest limits of this run — read before relying on it:

- 36 findings produced. **24 got a verdict** (15 survived, 9 refuted).
 - **12 findings never got a skeptic.** The refutation agents for the *operational* and *test-quality* lenses died on a session limit — the same failure mode that killed the August run. Those 12 are LEADS, not confirmed defects. They are listed separately below.
 - The refutation rate on the sample that *was* checked was 9/24 (38%), and two survivors were downgraded (high→low). Expect a similar fraction of the 12 to be noise.
-

Confirmed — survived adversarial refutation

[CRITICAL] Gate cannot see stale ciphertext: a plaintext value edited after the backfill passes as “ok” and 013 makes the stale value permanent

Where: server/scripts/verify-encryption.mjs:45–53, :79

Lens: gate

Mechanism. `countUnencrypted` asks only one question: `.not(plain,'is',null).is(enc,null)` — plaintext present AND ciphertext NULL. A row where BOTH are present but DISAGREE is invisible to it. The only thing that could catch disagreement is `sampleDecrypts`, and its comparison is guarded off entirely when the plaintext is NULL (line 79: `if (row[plain] !== null && row[plain] !== undefined && got !== String(row[plain]))`), so a NULL-plaintext row with a stale-but-valid ciphertext is reported “ok” even when it IS sampled. This is not hypothetical: no route imports `lib/crypto.js` (only `test/crypto.test.js`, `test/encrypt-backfill.test.js`, `scripts/encrypt-backfill.mjs` and this gate do), so EVERY app write between the backfill and 013 desynchronises the pair — `routes/transactions.js:432` writes `amount` alone, `routes/goals.js:175` writes `current_amount` alone, `routes/me.js:88` writes `monthly_limit` alone. The gate then certifies the drop of the only correct copy.

Failure scenario. Backfill runs Monday night. Tuesday morning a user corrects a mistyped transaction from £250 to £25 via PATCH `/api/transactions/:id` (`routes/transactions.js:432`). `amount=25`, `amount_enc` still decrypts to “250”. `countUnencrypted` counts 0 (`enc` is not null). The 50-row unordered sample almost certainly misses this row. Gate prints PASS. Migration 013 drops `amount`. The transaction is £250 forever, with no plaintext left to recover from. Variant hitting the line-79 guard: a user sets `monthly_limit=800`, is backfilled, then clears it (`routes/me.js:13` permits null) — after 013 the cleared limit silently returns as 800, and the gate would have said “ok” even if it had sampled that row.

[HIGH] Verification compares the DB against a stale in-memory snapshot, so a concurrent edit is silently “verified” as the old amount (*claimed critical, corrected to high*)

Where: server/scripts/encrypt-backfill.mjs:144-169 (re-SELECT), :110, :154

Lens: backfill

Mechanism. keysetScan fetches a page of up to 500 rows (`.limit(PAGE_SIZE)`, line 233), then `processRow` encrypts `String(row[plain])` from that snapshot (line 110). The post-write re-SELECT at line 144 requests ONLY the `_enc` columns (`job.fields.map(([, e]) => e)`) — it never re-reads the plaintext. Line 154 therefore compares the decrypted ciphertext to `row[plain]`, the value captured at page-SELECT time, not to what the plaintext column holds now. Because rows are processed serially with ~2 HTTP round-trips each, the last row of a page is written up to ~500x2 round-trips (order of a minute) after it was read. Any UPDATE to that plaintext in the gap is invisible: the check compares stale-to-stale and always passes. The row then has non-NULL `_enc`, so the `.is(firstEnc, null)` idempotency filter (line 231) hides it from every re-run, and `verify-encryption.mjs`’s `countUnencrypted` (line 44) does not flag it either (ciphertext is present) — only its unordered `.limit(sample)` spot-check (`verify-encryption.mjs:62-70`, default 50 rows) could catch it, by luck.

Failure scenario. Backfill is running over transactions during a “quiet period” (the header only asks for that, not a paused app). Alex opens the app and corrects a transaction from 50.00 to 75.00 — `routes/transactions.js:481` writes `payload.amount = 75`. The backfill had already read that row as 50.00 in the current page. It writes `amount_enc = E(“50”)`, re-reads `amount_enc`, decrypts “50”, compares to its stale `row.amount = 50` -> PASS, counted as “encrypted + verified against the database”. The row is now plaintext 75 / ciphertext 50. Re-runs skip it (`amount_enc` non-NULL). The gate passes. Migration 013 drops `transactions.amount` and the transaction is permanently 50.00. Same path applies to `savings_goals.current_amount` (`routes/goals.js:175` bumps it on every contribution) and `user_stats.monthly_limit`.

[HIGH] Vacuous success: `alreadyEncrypted` is unreachable on the keyset path, so “Backfill complete — 0 rows encrypted” is the success message for an empty or wrong database

Where: server/scripts/encrypt-backfill.mjs:285, :292-293, :273, :314

Lens: backfill

Mechanism. `counts.alreadyEncrypted` is only ever incremented at line 273, inside `offsetScan`. `offsetScan` is only reached when `pkOf(job).length > 1` (line 292-293), and every one of the seven entries in JOBS has a single-column PK (`transactions/budgets/savings_goals/savings_contributions/ask_messages/recurring_payments` default to `['id']`; `user_stats` declares `['user_id']`). `subscription_overrides` — the composite-PK table the `offsetScan` docstring names — is not in JOBS at all. So `offsetScan` is dead code and `alreadyEncrypted` is structurally always 0. Consequently `keysetScan` reports only rows it touched this run and never a denominator: it cannot distinguish “table already fully encrypted” from “table is empty” from “the `is null` filter matched nothing because I am pointed at the wrong project”. Line 314 emits `Backfill complete – 0 rows encrypted and verified against the database`. as the success message in all three cases, and exits 0.

Failure scenario. `server/.env` is pointed at the staging/empty Supabase project (two projects, one `.env`, a live-run day). `node scripts/encrypt-backfill.mjs` prints `transactions: 0 rows ...` for every table and `Backfill complete – 0 rows encrypted and verified against the database`. and exits 0. `verify-encryption.mjs` against the same URL prints PASS – nothing would be lost. 0 ciphertext

value(s) sampled and exits 0 (its count and sample are both trivially empty). The operator now has a green backfill and a green gate, and pastes migration 013 into the PRODUCTION SQL editor. Every plaintext money column on the live database is dropped with no ciphertext anywhere. Neither script ever prints a row count it expected to see.

[HIGH] PASS is not a snapshot, and nothing stops the 03:00 recurrences cron from inserting unencrypted rows into the gate’s blind window (*claimed critical, corrected to high*)

Where: server/scripts/verify-encryption.mjs:96–110, :116

Lens: gate

Mechanism. The gate issues 16 independent HTTP queries (countUnencrypted + sampleDecrypts per column pair, 8 pairs) at 16 different instants — there is no transaction, no snapshot, and no re-check. Its entire mitigation is a printed sentence at line 116 (“Migration 013 may proceed IN THIS WINDOW, with the app paused or read-only”): nothing pauses the app, nothing disables the cron, nothing records a timestamp 013 could be checked against. Meanwhile vercel.json schedules /api/cron/recurrences at 0 3 * * *, and lib/runRecurrences.js:58-66 inserts into transactions with a plaintext amount and no amount_enc (it cannot write one — it does not import lib/crypto.js). 03:00 is exactly the quiet hour an operator would choose for an irreversible migration.

Failure scenario. Operator runs the gate at 02:58 on a quiet night; it PASSes. At 03:00 Vercel Cron fires /api/cron/recurrences and runRecurrences.js:58 inserts six recurring rent/subscription transactions, each with plaintext amount and NULL amount_enc. At 03:05 the operator pastes migration 013 into the Supabase SQL editor and drops transactions.amount. Those six rows now have no amount in either column — total, unrecoverable loss of six real financial records, and the gate’s exit code 0 is what authorised it.

[HIGH] The gate’s scope is imported from the script it audits, so money columns outside JOBS are invisible — transactions.original_amount already is one

Where: server/scripts/verify-encryption.mjs:34; server/scripts/encrypt-backfill.mjs:78–89

Lens: gate

Mechanism. import { JOBS } from './encrypt-backfill.mjs' means the gate and the backfill share one scope definition, so the gate can never detect that the scope is wrong: a money column missing from JOBS is absent from the loop at lines 96-97, contributes nothing to missing or failures, and the run still prints “PASS — nothing would be lost” (line 115). Nothing compares JOBS against the live schema, and nothing compares it against migration 013’s DROP list (013 is unwritten, so the columns the gate checked and the columns 013 drops are joined only by an operator’s memory). Drift has already happened: migration 016 — applied to live, dated after the 2026-08-09 scope decision — added transactions.original_amount numeric(14,2), original_currency and fx_rate; routes/transactions.js:259-261 and :469-471 write them in plaintext; migration 012 adds no _enc for them and JOBS has no entry. lib/fx.js:46-55 defines convertToBase as Number((originalAmount * fxRate).toFixed(decimals)) — exactly the value stored in amount.

Failure scenario. Gate PASSes, 013 drops transactions.amount. Every foreign-currency transaction still carries original_amount and fx_rate in plaintext on the same row, and their product reproduces the dropped amount to the cent via the same formula the app used to compute it. For those rows the encryption is fully defeated against the exact threat it exists to stop (an attacker

with read access to the Postgres data), while the gate certified “nothing would be lost” — and any future money column added the same way will be equally invisible to this gate.

[HIGH] The only migration 013 on disk drops five plaintext columns that have no `__enc` twin, and the gate is structurally blind to all five (*claimed critical, corrected to high*)

Where: docs/superpowers/plans/2026-07-17-phase9-pln-privacy-history-pace-special.md:579,584,586,590,593

Lens: scope-coverage

Mechanism. Migration 013 is not written to disk, but a complete, copy-pasteable 013 body exists at plan lines 575-598, and BUILD_PLAN.md:752 explicitly directs the next session to read that plan document before doing this work. That SQL is from the PRE-RESCOPE scope: it drops `transactions.description` (:579), `categories.name` (:584), `savings_goals.name` (:586), `savings_contributions.note` (:590) and `subscription_overrides.display_name` (:593), then renames `description__enc/name__enc/note__enc/display_name__enc` into their places (:581,:585,:587,:592,:594). After the 2026-08-09 rescope, migration 012 (lines 32-54) creates NONE of those four `__enc` columns. Separately, `verify-encryption.mjs` derives its entire coverage from JOBS (imported at :34, iterated at :96-99), and JOBS (`encrypt-backfill.mjs`:78-89) contains no entry for `description`, `categories.name`, `savings_goals.name`, `note` or `display_name` — so the gate has no way to notice they are about to be dropped. It prints ‘PASS — nothing would be lost’ (`verify-encryption.mjs`:115) and authorises the run.

Failure scenario. Alex applies 012, runs the backfill, runs `verify-encryption.mjs`, sees PASS + ‘Migration 013 may proceed IN THIS WINDOW’, then pastes the plan’s 013 block into the Supabase SQL editor as instructed. `alter table public.transactions drop column amount, drop column description;` succeeds (both columns exist); the following `rename column description__enc to description` fails because `description__enc` was never created. 012’s own header (lines 27-30) states the author’s working assumption that a hand-pasted run ‘fails halfway through’ and leaves partial effects — under that assumption every merchant description for 5 real users is permanently gone, along with `categories.name`, `savings_goals.name`, `savings_contributions.note` and `subscription_overrides.display_name`, none of which any ciphertext exists for. Even in the best case where the editor wraps the script in one transaction and rolls it all back, the gate demonstrably certified as safe a script that would have destroyed data.

[HIGH] The nightly recurrences cron writes plaintext money into transactions and never populates `amount__enc`

Where: server/lib/runRecurrences.js:58-67

Lens: scope-coverage

Mechanism. `processOne` inserts `{ user_id, category_id, amount: row.amount, type, description, date, is_recurring, recurrence_id }` — plaintext amount only, no `amount__enc`, and it does not import `lib/crypto.js`. It reads `row.amount` from `recurrences` via `RECURRENCE_COLUMNS` (:14-15), which selects the plaintext amount. The plan’s 013 (plan:580) renames `transactions.amount__enc` to `amount`, making that column TEXT with no check (`amount > 0`) (the original constraint dies with the dropped numeric column, `migrations/001_init.sql`:61). PostgREST will happily insert the numeric literal into a text column. Nothing in Phase 9.5 touches this file.

Failure scenario. Pre-013: the cron fires at 03:00 the night after Alex runs the backfill and the gate. It inserts a rent transaction with `amount=1200.00` and `amount__enc NULL`. If 013 runs in

that window on the strength of the earlier PASS, `drop column amount` destroys that £1200 row's value outright — the row survives with no recoverable amount. Post-013: the cron keeps inserting `amount: 1200.00` into what is now a TEXT ciphertext column; the write succeeds silently, and the first dashboard/analytics read that calls `decryptField` on '1200.00' throws (it is not a `v1:` envelope), 500-ing that user's Dashboard on a row nothing in the app can repair.

[MEDIUM] No durable in-flight marker: any interruption between the write and the rollback strands a row that no re-run can ever see (*claimed high, corrected to medium*)

Where: `server/scripts/encrypt-backfill.mjs:132-176` and `:196-211` (`rollbackEnc`), interacting with `:231`

Lens: backfill

Mechanism. `rollbackEnc` is an in-process compensating write. It only runs if the Node process survives long enough to execute it. Nothing durable records “I am mid-write on table T pk P” — there is no journal, no in-progress flag column, no per-row status. The moment the update at `:134` commits, the row leaves the `.is(firstEnc, null)` filter, which is the script's ONLY record of what still needs doing. `rollbackEnc`'s own docstring correctly identifies this failure mode for error paths but the same state is reachable with no error at all, and the header's advertised mitigation (“a re-run is cheap, idempotent, and picks up any straggler”, `:41`) is false for exactly these rows. The run's duration makes the trigger likely: 7 tables processed serially, roughly two sequential HTTP round-trips per row (`:134` and `:146`), so a few thousand rows is tens of minutes with no progress bar and no resume — an operator Ctrl-C is a normal event, not an exotic one.

Failure scenario. Operator starts the backfill, sees it grinding with no output for 20 minutes, presses Ctrl-C (or the laptop sleeps / VPN drops) while awaiting the re-SELECT at line 146. Node exits; `rollbackEnc` never runs. That row is committed with unverified ciphertext. Re-run skips it (filter at `:231`). `verify-encryption.mjs`'s count check does not flag it and its `unordered.limit(50)` sample (`verify-encryption.mjs:62-70`) will not reach it in a 12k-row table. Gate PASSes, 013 drops the plaintext. If the write was fine, no harm — but the whole point of steps 3-4 is that the script does not get to assume that, and after 013 there is no way to find out.

[MEDIUM] A single concurrent DELETE by any live user aborts the whole seven-table sweep with a message that reads like data corruption

Where: `server/scripts/encrypt-backfill.mjs:132-134`, `:146-150`

Lens: backfill

Mechanism. The update at `:132-134` is not checked for rows-affected — PostgreSQL returns no error when an UPDATE matches zero rows, and the script does not request a representation. The only signal that the row is gone is `!stored` from the re-SELECT at `:146-148`, which throws `VERIFY FAILED` (row not found on re-read after write) (`:149`). That exception propagates through `rollbackEnc`, out of `processRow`, out of `runJob` and `runBackfill` (no per-row try/continue anywhere), to `main()`'s `process.exit(1)` at `:354`. There is no distinction between “this row was deleted” (benign, expected on a live app) and “the database did not store what I wrote” (the alarming case the message describes).

Failure scenario. Backfill is 40 minutes into a maintenance window. One of the five users deletes a transaction in the UI (`routes/transactions.js:502`) — or deletes an Ask Trim message (`routes/ask.js:52`), or a budget (`routes/budgets.js:185`), or a goal (`routes/goals.js:132`) — after the

backfill's page SELECT read it but before the row is processed. The update silently matches zero rows, the re-read returns nothing, the script prints **VERIFY FAILED (row not found on re-read after write) transactions id=<uuid>** and exits 1 with the whole remaining sweep (`savings_*`, `user_stats`, `ask_messages`, `recurrences`) untouched. The operator's reasonable reading of that message is that Postgres dropped their write, so the window is spent on investigating a phantom corruption bug instead of on the migration.

[MEDIUM] count ?? 0 makes the completeness check fail OPEN when PostgREST returns no count (*claimed high, corrected to medium*)

Where: `server/scripts/verify-encryption.mjs:52`

Lens: gate

Mechanism. `postgrest-js` sets `count = null` while leaving `error = null` whenever the count cannot be read: `node_modules/@supabase/postgrest-js/dist/index.mjs:341` initialises `count = null`, and `:354-355` only assigns it if `(countHeader && contentRange && contentRange.length > 1)` — i.e. an otherwise-successful 200/206 whose `content-range` response header is absent or has no / yields `{error: null, count: null}`. `return count ?? 0` then converts “I could not count” into “zero rows are at risk”, which is the PASS answer. The same fail-open default appears at lines 70 and 83 (`data ?? []`), where a null body is reported as “0 sampled, 0 bad” — verdict ok. A gate authorising an irreversible drop must treat an unanswered question as FAIL, not as zero.

Failure scenario. The operator runs the gate through a corporate proxy, a Supabase edge/CDN hop, or any layer that strips the non-safelisted `Content-Range` response header. All 8 `countUnencrypted` calls return `{error: null, count: null}` → each returns 0 → `missing` stays 0. The 50-row samples return rows that decrypt fine (they were backfilled correctly weeks ago). The gate prints [ok] for all 8 pairs and “PASS — nothing would be lost”, having established nothing whatsoever about completeness. Migration 013 drops plaintext across seven tables on the strength of eight unanswered queries.

[MEDIUM] --sample=N and every mistyped flag are silently ignored, so a deliberate deep check runs at depth 50 and still exits 0

Where: `server/scripts/verify-encryption.mjs:127-130`

Lens: gate

Mechanism. `const i = argv.indexOf('--sample')` matches only the exact standalone token. The equals form `--sample=2000` never matches, `i` is -1, and `sample` falls back to `DEFAULT_SAMPLE` (50) with no warning. Unlike its sibling — `encrypt-backfill.mjs:327-337` explicitly rejects unknown flags precisely because “an unrecognised flag would otherwise perform a LIVE run against real data” — this script has no unknown-argument check at all, so `--samples 2000`, `-s 2000`, `--sample-size 2000` and any typo are all accepted silently and the run still exits 0. `Number(argv[i + 1])` also accepts fractional values (`--sample 2.5` passes both `Number.isFinite` and `> 0`).

Failure scenario. Before the irreversible drop the operator deliberately asks for deeper coverage: `node scripts/verify-encryption.mjs --sample=2000`. `indexOf` returns -1, the gate samples 50 rows per column pair, prints “...50 sampled...” in small type among 8 output lines, then “PASS”, then exits 0. The operator believes 2,000 rows per column were verified and proceeds to migration 013 with 2.5% of the coverage they asked for — and with the least coverage over exactly the recently-written rows the deep check was meant to reach.

[MEDIUM] `transactions.original_amount + fx_rate` are money columns in none of the three lists, and together they reconstruct the encrypted amount exactly (*claimed high, corrected to medium*)

Where: `server/migrations/016_foreign_currency.sql:21-31`

Lens: scope-coverage

Mechanism. Migration 016 added `transactions.original_amount` `numeric(14,2)`, `original_currency` text and `fx_rate` `numeric(18,8)` on 2026-08-10. Migration 012 (lines 32-33) adds only `amount_enc` for transactions, JOBS (`encrypt-backfill.mjs:79`) encrypts only `['amount', 'amount_enc']`, and 012's 'dropped from scope, deliberately' enumeration (lines 11-22) never mentions these columns at all — they are unconsidered, not excluded. `lib/fx.js:54` defines the stored value as `Number((originalAmount * fxRate).toFixed(decimals))`, and `routes/transactions.js:259-261` and `:469-471` persist all three alongside it. So the plaintext pair is not merely an extra leak: it is a closed-form recovery of the ciphertext's contents.

Failure scenario. Alex logs a €45.00 dinner in Paris at rate 0.85565. Row stores `amount=38.50` (encrypted to `amount_enc` after backfill, dropped by 013), `original_amount=45.00`, `original_currency='EUR'`, `fx_rate=0.85565` — all three in plaintext, forever. Anyone with read access to the Supabase table (the exact adversary Phase 9.5 exists to defend against) computes $45.00 * 0.85565 = 38.50$ and recovers the supposedly-encrypted amount for every foreign-currency transaction, with no key. `verify-encryption.mjs` prints PASS because `original_amount` is not a JOBS pair and is therefore never examined.

[LOW] `decryptField` error message echoes the stored value verbatim, and the global error handler logs it (*claimed high, corrected to low*)

Where: `server/lib/crypto.js:72`

Lens: crypto-core

Mechanism. `throw new Error(Unknown ciphertext version: ${parts[0]})` interpolates everything before the first colon of the stored value. On the exact state the header anticipates at `crypto.js:9-11` (bare plaintext left behind by a half-finished backfill), `parts[0]` is the user's plaintext. `server/index.js:120` logs `err.message` for every error reaching the route error handler, so it lands in persistent Vercel logs. Both existing callers already defend against this — `encrypt-backfill.mjs:159-160` and `verify-encryption.mjs:76` discard the message with 'Never echo the value — it is user data' — proving the hazard is known, but the leak lives in the module, and the ~7 unwritten routes will not remember. The other three messages (parts count, IV length, tag length) leak only integers, so this is the single leaking line.

Failure scenario. Backfill is interrupted (its own header, `encrypt-backfill.mjs:39-41`, admits stragglers). `ask_messages.content_enc` for one row still holds raw content — a column that migration 012:48 describes as free text that 'could contain anything', capped at 8000 chars by 007:19. A route decrypts it, `decryptField` throws, `index.js:120` writes the entire message body (up to the first colon, i.e. usually all 8000 chars) into the server log in cleartext. Verified: `decryptField(U, 'I spent 4200 on IVF treatment at Care Fertility')` throws with that whole sentence in `err.message`. The at-rest encryption feature thereby copies the plaintext it was built to hide into a second, unencrypted store.

[LOW] Master key is accepted with no entropy requirement, and its custody contract points at a SECURITY.md section that does not exist (*claimed medium, corrected to low*)

Where: server/lib/crypto.js:44

Lens: crypto-core

Mechanism. masterKey() validates shape (crypto.js:38), decoded length (:44) and base64 canonicity (:49) — carefully — but never that the 32 bytes came from a CSPRNG. Verified: base64('passwordpasswordpasswordpassword') and 32 zero bytes are both accepted and encrypt normally. The thoroughness of the surrounding checks reads as though the key is validated. Compounding it, crypto.js:6 states 'Losing DATA_ENCRYPTION_KEY = losing every user's data. See SECURITY.md' and encrypt-backfill.mjs:45 repeats 'backed up DATA_ENCRYPTION_KEY (see SECURITY.md)' — but grep shows DATA_ENCRYPTION_KEY appears nowhere in SECURITY.md. Its 'Key separation (the most important rule)' table (SECURITY.md:9-20) enumerates all seven other keys and omits this one, and its threat model (SECURITY.md:7) never mentions at-rest data or the encryption feature at all. The generate-and-back-up instruction survives only in BUILD_PLAN.md:754 and the plan/spec docs.

Failure scenario. Alex provisions the server (or a second environment, or restores after a laptop loss) working from SECURITY.md, the document CLAUDE.md designates as the non-negotiable auth/data contract and that crypto.js itself redirects him to. DATA_ENCRYPTION_KEY is not listed there, so it is not in the backup checklist (SECURITY.md:136 covers only rotating the service-role key). Post-013 the plaintext is gone and the sole copy of the key lives in a Vercel env var; losing that dashboard entry, or replacing it, permanently destroys five people's financial history. Separately, an operator satisfying the error text's literal demand ('must be 32 bytes base64') by base64-ing a 32-character passphrase gets a brute-forceable key that the module accepts silently, voiding the at-rest guarantee while every check passes.

[LOW] subscription_overrides.merchant_key stores recurring charge amounts in a plaintext primary key (*claimed medium, corrected to low*)

Where: server/lib/subscriptions.js:85-91

Lens: scope-coverage

Mechanism. For subscriptions detected without a description, syntheticMerchantKey builds auto:<categoryId|none>:<bucket>:<cadence> where bucket = bucketAmount(amount) = Math.round(amount/5)*5 (:85-87). routes/subscriptions.js:255 and :264 persist that string as subscription_overrides.merchant_key, which is half the table's primary key (migrations/004_subscription_overrides.sql:13). The table appears in none of the three lists: not in migration 012, not in JOBS, and 012's deliberate-exclusion list (lines 11-22) names only subscription_overrides.display_name — the amount-bearing merchant_key is not mentioned anywhere.

Failure scenario. Alex audits a £47/month gym charge that his bank exports without a description. The detector writes merchant_key='auto::45:monthly'. After the full backfill and migration 013, every transaction amount in the database is ciphertext — but this row still says, in plaintext and in a column that cannot be encrypted without rewriting the primary key, that this user pays roughly £45 every month in that category. Repeated across his subscriptions it reconstructs a monthly outgoings profile from a table the encryption work never looked at.

Refuted — investigated and dismissed

Recorded so they are not re-raised. Each was checked against the real code by an agent trying to disprove it.

No AAD: ciphertext is freely replayable across columns, rows and tables within the same user

Lens: crypto-core

The technical mechanism is accurate and I reproduced it: `server/lib/crypto.js:62` and `:90` call `createCipheriv/createDecipheriv` with only `{authTagLength}` and no `setAAD` (grep for `setAAD` across `server/` hits only `node_modules/@types/node` and `node_modules/jose` — zero project source). The only bound context is the user, via `hkdfSync(..., user:${userId}, ...)` at `crypto.js:56`. Executing it, a ciphertext produced for `savings_goals.target_amount_enc` decrypted cleanly as `current_amount_enc`, and a `budgets.amount_limit_enc` value decrypted as `transactions.amount_enc`; cross-user replay correctly failed, so the header claim at `crypto.js:3-5` holds as written.

But the claim fails as a defect, on four grounds:

- 1) Out of the stated threat model. `SECURITY.md:7` scopes the threat to “another logged-in user trying to read someone else’s data,” plus credential stuffing, XSS and DoS. A write-capable DB attacker appears nowhere. The claim leans on `crypto.js:12-15`, but that comment justifies the 16-byte tag assertion — it concerns forging a NEW ciphertext for a CHOSEN value, a strictly stronger capability than replaying an existing one. It is not a promise of positional integrity.
- 2) No regression versus the status quo; arguably an improvement. `savings_goals.current_amount` is a plain numeric column today. Anyone with DB write access can already set `current_amount := target_amount`, or `transactions.amount := 1`, silently and irreversibly, with MORE freedom than the post-encryption attacker: any value at all, versus only values already encrypted under that user’s key. The absence of AAD does not create the failure scenario — it declines to fix one that predates the feature while shrinking the attacker’s reachable value set.
- 3) The claimed pre-013 comparand is overstated. `verify-encryption.mjs:79` sits inside `sampleDecrypts`, bounded by `.limit(limit)` at `:66` with `DEFAULT_SAMPLE = 50 (:36)`. It is a one-shot gate sampling at most 50 rows per column, not a runtime integrity signal — it would catch a tampered row only by chance. The claim’s before/after contrast (“pre-013 the plaintext would expose the mismatch”) therefore misrepresents what the code does.
- 4) Unreachable by any code path in the repo. No route imports `lib/crypto.js` (importers are only `server/test/crypto.test.js:3`, `server/test/encrypt-backfill.test.js:8`, `verify-encryption.mjs:33`, `encrypt-backfill.mjs:50`). The sole writer of `_enc` columns is `encrypt-backfill.mjs:110`, which derives each `_enc` from its own paired plaintext inside `for (const [plain, enc] of job.fields)` and then re-reads and compares per column at `:153-170`, throwing at `:165` on any mismatch. The “buggy UPDATE” variant has no candidate UPDATE to cite.

Additionally, a partial guard narrows the blast radius the claim asserts: `decryptAmount` (`crypto.js:106-115`) rejects non-finite decrypted plaintext, so `ask_messages.content_enc` cannot be

replayed into any amount column (verified — it throws). Replay is confined to the six amount columns, not “all seven columns’ ciphertexts are interchangeable.”

What genuinely survives is a hardening note, not a defect: AAD binding table|column|pk is cheap, and retrofitting after the backfill would require re-encrypting every row, so the decision is worth making before migration 012 is applied. The v1 prefix (crypto.js:22, checked at :72) makes a v2 envelope feasible. That is design timing, not a bug in behavior — hence low, not critical.

The amount fail-closed rules never run on any database path, so the migration-013 gate certifies under weaker rules than the runtime enforces

Lens: crypto-core

REFUTED as stated. The factual observations are correct, but the causal chain to the claimed harm does not exist, and the proposed remedy would not change the outcome even if it did.

What is TRUE (verified): - `encryptAmount/decryptAmount` (server/lib/crypto.js:95, :106) are imported only by server/test/crypto.test.js:3. Nothing else in server/ or client/ imports them (repo-wide grep). - The backfill writes every amount column with `encryptField` (server/scripts/encrypt-backfill.mjs:110) and verifies with `decryptField` (:124, :157); the 013 gate uses `decryptField` (server/scripts/verify-encryption.mjs:73).

Why the failure scenario does not hold, line by line:

1. The only existing writer cannot produce a non-finite amount. `processRow` encrypts `String(row[plain])` where `row[plain]` is read straight out of the plaintext column (encrypt-backfill.mjs:106-112). Every amount column in JOBS is `numeric(14,2)` not null check (`amount > 0`) — `001_init.sql:61, :79, :94, :96, :105` and `014_recurrences.sql:17` — and `user_stats.monthly_limit` is `numeric` null check (`monthly_limit is null or (monthly_limit > 0 ...)`) (`008_monthly_limit.sql:5-6`). A NULL is short-circuited to `patch[enc] = null` at :107-108. So 'undefined', 'NaN' and '' are unreachable on the only path that has ever been coded. Nothing can seed such a value either: `routes/transactions.js:41` and :73 validate amount with `z.number().positive().finite().max(1_000_000_000)`.
2. The “idiom the only existing writer establishes” premise is contradicted by the written plan. `docs/superpowers/plans/2026-07-17-phase9-pln-privacy-history-pace-special.md:568` (Step 6, Route sweep) specifies literally `amount_enc: encryptAmount(req.user.id, amount)` and reserves `encryptField` for the text column. The unwritten route sweep is specified to call the validating helper, not the one the backfill uses.
3. The gate’s plaintext comparison is strictly STRONGER than the amount rule, in exactly the window the gate runs. `verify-encryption.mjs:15-21` defines it as the pre-013 gate (“run 013 ONLY if this exits 0”). Pre-013 the plaintext column is still there, so for every sampled row line :79 asserts `got === String(row[plain])` against a numeric not null check (`>0`) value — that subsumes `Number.isFinite`. A row whose `amount_enc` held 'undefined' while `amount` held the real number would be reported as “decrypts to something else” (:80), not passed.
4. The claim’s escape hatch — “post-013 there is no plaintext to compare to at :79” — describes running the gate after the migration it exists to authorise. In that state `countUnencrypted` (:45-52) selects the dropped column, `PostgREST` returns an error, `if (error) throw error` propagates out of `verifyEncryption`, and `main()`’s catch exits 1 (:145-148). Post-013 the gate cannot exit 0 at all, so it cannot certify anything.

5. Decisive: the proposed fix has no effect on the only residual gap. The gap that actually exists is sampling coverage — `sampleDecrypts` pulls `.limit(50)` unordered (:60-66), so a bad row among thousands may never be examined. But `decryptAmount` would run inside that same loop on that same sample. For every row the gate looks at, :79 already catches more than a finiteness check; for every row it does not look at, `decryptAmount` would not run either. Swapping the helper changes nothing about the claimed data-loss path.

Genuine residual (narrower than claimed, and not the claimed mechanism): for a row where the plaintext amount is NULL but the `_enc` column holds a valid envelope of non-numeric text, :79 skips the comparison (`row[plain] !== null` guard) and `decryptField` succeeds, so the gate passes it. Among amount columns only `user_stats.monthly_limit` is nullable (008:5-6), and no code path currently writes an `_enc` value without the plaintext beside it (`encrypt-backfill.mjs:107-108` writes NULL for NULL). This is a hardening observation about a dead code path, not a live loss scenario.

Severity: claimed high is inflated. The feature is inert (no route imports `lib/crypto.js`), the harm requires a typo in code that has not been written and that the plan tells the author to write with `encryptAmount`, and the gate would catch it anyway while plaintext exists. Corrected to low — worth a note that the header's amount rule at `crypto.js:16-18` is currently enforced by nothing that touches a database, so the route sweep must be held to plan line 568.

A NULLED ciphertext column is indistinguishable from a legitimately absent value, and 013 removes the NOT NULL/CHECK constraints that used to make it impossible

Lens: crypto-core

Refuted at both layers it claims, and the residual is a note on an unwritten file.

CRYPTO LAYER — the behavior is required, not a hole. `decryptField` returns null for null (`server/lib/crypto.js:68`) and `decryptAmount` propagates it (:107-108); I executed it and confirmed `decryptAmount('u', null) === null` with no throw and no `DATA_ENCRYPTION_KEY` set. But `encryptAmount` is symmetric (:96), and the JOBS list contains a legitimately-nullable money column: `user_stats.monthly_limit` is numeric null check (`monthly_limit is null or ...`) (`server/migrations/008_monthly_limit.sql:5-6`), NULL for every user until they set a limit — the backfill header calls this out at `encrypt-backfill.mjs:14-17`. Making `decryptAmount` throw on null (the claim's implied fix) would 500 GET /api/me for every user without a limit. Additionally `decryptAmount` has zero non-test callers repo-wide: `encrypt-backfill.mjs:50` and `verify-encryption.mjs:33` import only `decryptField`, and no route imports `lib/crypto.js` at all.

DB LAYER — 012 is correct as written and 013 does not exist. 012:32-54 does create bare nullable text, and the plaintext constraints are real (`001_init.sql:61 amount numeric(14,2) not null check (amount > 0)`, :79, :94, :105; `007_ask_messages.sql:19`). But 012 MUST create them nullable — the backfill writes NULL for NULL sources (`encrypt-backfill.mjs:107-108`) and populates rows progressively; a NOT NULL at 012 time would make the whole dual-write phase impossible. Nothing in 012 prevents adding NOT NULL in 013. Migration 013 is not written (grep confirms: only prose drafts). The only draft SQL is `docs/superpowers/plans/2026-07-17-phase9-pln-privacy-history-pace-special.md:575-597`, and that same plan already instructs the author at ~:599 to check 001 for constraints on dropped columns, naming amount CHECK constraints specifically. A defect cannot live in unwritten SQL whose own plan already flags the review.

ROLLBACK VECTOR — refuted outright. Pre-013, `rollbackEnc` (`encrypt-backfill.mjs:196-211`) NULLs only the `_enc` columns; the plaintext column is never written by this script (:31-33). Noth-

ing is lost — that is the rollback’s purpose. Post-013 the script cannot reach `rollbackEnc` at all: `keysetScan` builds `cols` from plaintext column names (:223) and filters `.is(firstEnc, null)` (:231); under the planned 013 (drop plaintext, rename `_enc` to the original name) both references are to columns that no longer exist, so PostgREST returns 42703 and `runJob` throws before any row is processed. “Partially-applied UPDATE” is also not a state PostgREST can produce — each update is a single atomic statement.

ATTACKER MODEL — the constraint was never a security control. `001_init.sql:200-217` and `002_service_role_grants.sql:10-17` grant table privileges to `service_role` only; authenticated/anon get no grants through the Data API (RLS policies at `001:141/148` are defence-in-depth over a role that has no grant). So the only write-capable actor is a service-role-key holder, who can DELETE the row outright — a deletion primitive requiring no encryption key either. The claim’s central framing (“the one deletion primitive available to a write-capable attacker is the one the design does not fail closed on”) is false.

EXISTING GUARD — `verify-encryption.mjs:45-53` counts exactly `plaintext is not null AND enc is null` for every JOBS field and gates 013 on that count being zero (:112-122). The NULL-ciphertext state is precisely what the completeness gate refuses.

REACHABILITY — 012 never applied, 013 unwritten, backfill never run, `DATA_ENCRYPTION_KEY` unset, no route imports `crypto.js`. There is no code path today, and the post-013 path the claim describes requires route code (plan Step 6) that does not exist, so “no exception raised anywhere” is an assertion about unwritten consumers.

RESIDUAL (low, and not at the claimed location or mechanism): when 013 is eventually written, it should carry `not null` onto the renamed amount columns and a `char_length(content) > 0` onto `ask_messages.content`, since the value-referencing checks (`> 0`) genuinely cannot survive on ciphertext. That is one line on the 013 checklist, not a medium defect in `crypto.js:107`.

No key identifier in the envelope and no dual-key read path; a wrong master key is indistinguishable from tampering

Lens: `crypto-core`

REFUTED. The claim’s surface observations are accurate, but the two mechanisms it rests on are false against the real code, and its proposed fix does not address the harm it describes.

WHAT IS TRUE (verified): - `server/lib/crypto.js:64` emits exactly `[VERSION, iv, tag, ct]`; HKDF info is only `user:${userId}` (`crypto.js:56`); `decryptField` (`crypto.js:67`) takes no key parameter and reaches `process.env` via `masterKey()` (`crypto.js:32`). - I ran it: a row encrypted under key A and decrypted under key B throws “Unsupported state or unable to authenticate data”, byte-identical to the message a bit-flipped ciphertext produces under the correct key. So “wrong key and tampering share an error string” is real. - There is no rotation script and no rotation runbook. `grep` for `rotat|keyid|key` generation across the repo returns nothing for `DATA_ENCRYPTION_KEY` (only `jose/JWKS` and a CSS `rotate`).

WHAT IS FALSE: 1. “`decryptField ...` reads `process.env` directly, so a process cannot hold two generations at once.” The opposite is true, and it is true *because* of the direct `env` read. `masterKey()` (`crypto.js:32`) reads `process.env` on EVERY call — it is not cached at module load — so one process can swap generations between calls. The repo already demonstrates this pattern: `server/test/crypto.test.js:11-21` `withKey()` swaps `DATA_ENCRYPTION_KEY` mid-process and `crypto.test.js:52-71` rely on it passing. My probe built a working dual-key reader on top of the

unmodified module and read both an old-generation and a new-generation row in a single process (old row -> “42.50” via gen old; new row -> “99.00” via gen new). A rotation script is writable today against crypto.js as it stands, with zero changes to the envelope.

2. “no way to tell which rows are under which key mid-rotation.” The GCM auth tag IS that oracle, and it is the same information a key id would carry. Trial decryption (try new key, on auth failure try old) is deterministic: 200/200 wrong-key attempts failed, none silently succeeded, and GCM’s false-accept bound is 2^{-128} . A key identifier would save one failed decryption per row, not enable something otherwise impossible.
3. The stated harm does not follow from the stated defect. In the scenario “Alex restores the wrong key from ~/Keys/”, a key identifier would print “this row needs generation 1, you loaded generation 2” — it would not produce generation 1. If the correct key still exists, rotation and recovery work today by trial decryption (point 2). If the correct key is genuinely lost, the data is gone whether or not the envelope names it. The claim attributes an unrecoverable-data outcome to a missing label that cannot prevent it.
4. “Alex cannot tell whether the env var is wrong (data fine) or the DB is corrupt (data gone).” He can, without any code change: hold the candidate key and attempt one row — success means right key; and a uniform failure across all 5 users and all 7 JOBS tables (encrypt-backfill.mjs:78-90) is not a corruption signature, since corruption is localized. Trial-decrypting a single row against each backup copy of the key settles it definitively.

FAIL-CLOSED, NO WRONG OUTCOME: in both the wrong-key and the tampered case the module throws (crypto.js:92 via decipher.final()). Nothing is returned, nothing partially trusted, nothing written. There is no incorrect value reaching a user or a total — only a less specific error string. That is diagnostics, not a defect with a concrete wrong outcome.

REACHABILITY: the scenario needs migration 013 (does not exist — `ls migrations | grep 013` is empty), routes wired to crypto.js (grep over routes/, lib/, middleware/, index.js finds zero importers), `DATA_ENCRYPTION_KEY` set (absent from .env and .env.example), and the backfill run (never run). Every one of those steps is where a rotation runbook would be authored, and authoring it then requires no change to the envelope format. The design spec at docs/superpowers/specs/2026-07-17-pln-privacy-history-pace-special-design.md:109 already names the v1 prefix as the rotation slot, and decryptField’s version check (crypto.js:72) is the dispatch point a v2 generation would hook into — rows are already self-describing by version.

RESIDUAL (below the bar, noted for honesty): the dual-key read pattern requires mutating a process-global, which is safe for a single-threaded batch rotation script but would race if ever attempted inside the live Express request path. An optional key parameter on decryptField would be tidier. That is a forward-looking design preference for code not yet written, not a traced failure, so I did not upgrade it to a finding.

An errored UPDATE bypasses rollbackEnc entirely — the one case where commit status is genuinely unknown is the only one not rolled back

Lens: backfill

The structural facts are correct but the critical consequence does not follow, so the finding as written is refuted.

VERIFIED AS STATED: encrypt-backfill.mjs:134-135 (`const { error: upErr } = await update;`

`if (upErr) throw upErr;`) is outside the try at :140-174, so `rollbackEnc (:172)` is not reached for an errored UPDATE. `keysetScan`'s rerun filter `.is(firstEnc, null)` at :231 would skip a row left with non-null ciphertext, and `verify-encryption.mjs` `countUnencrypted (:41-50, plaintext not-null AND enc null)` would not count it. No guard elsewhere covers the path; `runBackfill (:307)` just propagates. Tests cover only the read-error rollback (`test/encrypt-backfill.test.js:142`); the fake client's update never errors (:61).

WHY THE CRITICAL HARM IS UNREACHABLE: the claimed loss requires a write that committed AND stored something other than what was sent ("if that write was truncated/mangled the amount is unrecoverable"). That branch does not exist here: 1. Every `_enc` column is unbounded text (`migrations/012_encryption_columns.sql:33-54`); `grep -niE 'varchar|char('` across all of `server/migrations/` returns nothing. There is no truncating column. 2. The stored envelope is `v1:<b64>:<b64>:<b64>` — pure ASCII, no NUL (`lib/crypto.js:64-68`). Nothing for a UTF-8 text column to mangle. 3. The PATCH is a single-row statement pinned by PK (:132-133). Postgres applies it atomically. A body truncated in transit yields invalid JSON, so PostgREST 400s and nothing commits — there is no "committed but mangled" outcome to produce.

That leaves only two real post-upErr states, both safe: - Did not commit: `_enc` stays NULL, so the rerun filter at :231 INCLUDES the row, and even without a rerun `verify-encryption`'s `countUnencrypted` returns `>0` and the gate FAILS. The genuinely lossy case is exactly the one the gate can see. - Did commit: the stored bytes are byte-identical to the ciphertext already proved correct in memory at :122-127 (`decryptField(patch[enc]) === String(row[plain])`). Migration 013 dropping that row's plaintext loses nothing — the ciphertext decrypts to the original amount.

The sample-of-50 argument in the claim is therefore moot: there is no bad ciphertext for the sample to miss on this path.

RESIDUAL REAL GAP (why not "not-a-defect"): the row does skip the database round-trip verification, and — unlike the rollback-failure path, which prints an explicit "!! do NOT run migration 013" banner (:205-209) — the operator sees only a raw PostgREST error, while the script header (:39-41) tells them a rerun is "cheap, idempotent". That is an operator-visibility inconsistency worth fixing (move the throw inside the try, or emit the same warning on upErr). It is not irreversible data loss.

Scenario reachability caveats noted but not load-bearing: the script has never been run, migration 012 has never been applied, `DATA_ENCRYPTION_KEY` is unset, and no route imports `crypto.js` — so nothing is at risk today. I did not lean on that to refute, since the audit is precisely about readiness for a live run.

The test suite exercises neither the update-error path nor any multi-column job, so both rollback gaps ship green

Lens: backfill

The coverage facts are accurate but the defect is not. Verified line by line:

ACCURATE: `makeFake` injects only `faults.selectError` (`test/encrypt-backfill.test.js:42`) and `faults.corruptColumn (:57)`; the update branch (:61) always resolves `{error:null}`, so `encrypt-backfill.mjs:135` (`if (upErr) throw upErr`) and `rollbackEnc`'s catch (:204-210) are never executed. All tests use the single-field JOB (:76) or the single-field `user_stats` job (:111); `savings_goals` (`encrypt-backfill.mjs:81`), the only two-field job, is never run. The fake sorts by `r.id` only (:65), and the two `user_stats` rows take the nothing-to-encrypt path on a short first page, so `.gt(cursorCol,`

cursor) (encrypt-backfill.mjs:234) is never exercised for a non-id PK. (Minor error in the claim: the suite has NINE tests, not ten — confirmed by `node --test.`)

REFUTED, three independent ways:

1. No defect exists in the code as written. `rollbackEnc` NULLs every field unconditionally (encrypt-backfill.mjs:198: `for (const [, enc] of job.fields) patch[enc] = null;`). I built a throwaway multi-column harness for `savings_goals` against the real `runBackfill`: both columns encrypt and verify correctly, and corrupting `current_amount_enc` rolls back BOTH columns to NULL. The untested path is correct, so the gap hides nothing.
2. The claimed consequence is factually wrong. The claim says the partial-rollback state “verify-encryption.mjs will only notice if that row lands in its 50-row unordered sample.” `verifyEncryption` iterates per column pair (verify-encryption.mjs:97) and calls `countUnencrypted` (:45-53), which is an EXACT count: `'exact', head:true` over `.not(plain,'is',null).is(enc,null)` for each pair — not a sample. `savings_goals.current_amount` is `numeric(14,2)` not `null` default `0` (001_init.sql:95), so it is never NULL and always trips that exact count. I ran the described post-regression row (`target_amount_enc` populated, `current_amount_enc` NULL) through the real `verifyEncryption`: `missing=1, pass=false`, “DO NOT RUN MIGRATION 013”. The gate blocks it deterministically, so the irreversible-loss chain does not close.
3. The trigger is hypothetical, not a state reachable from current inputs. It requires a future edit (“someone tightens `rollbackEnc`”, “moves throw upErr”). The audit bar requires specific inputs/state producing a specific wrong outcome; none exists against this code.

Also checked the sub-claim that `throw upErr` sitting outside the try is itself a gap: a single-row Postgres UPDATE is atomic, so a returned error means nothing committed and there is nothing to roll back. The only residual case (client timeout on a write that landed) leaves VALID ciphertext, which the gate’s sample passes anyway. Not a defect.

What remains is a real but low-value regression-suite gap in a script gating an irreversible migration: an `updateError` fault and one two-column job test would cost ~20 lines and are worth adding. Blast radius is bounded by `verify-encryption.mjs`’s exact per-column count, so medium is inflated.

sampleDecrypts uses LIMIT with no ORDER BY, so it re-checks the same oldest rows every run and gives no coverage over the rows actually at risk

Lens: gate

The syntactic observation is correct (`verify-encryption.mjs`:62-66 has `.limit()` with no `.order()`), so row order is undefined and will repeat in practice), but the claimed consequences do not follow.

1. The gate’s completeness assurance is NOT sampled. `countUnencrypted` (verify-encryption.mjs:45-53) is an exact count: `'exact', head: true` over the entire table filtered `plain is not null AND enc is null`. That count feeds `missing` (line 100) and the `pass` condition (line 112). “PASS — nothing would be lost” (line 115) rests on a full-table count, not on 50 rows.
2. Two of the three at-risk populations the claim names are covered by that exact count and not by the sample at all: `cron` inserts into `recurrences` (encrypt-backfill.mjs:88) and rows written behind the backfill cursor both appear as `plaintext-present/enc=NULL`, exactly the predicate at lines 49-50. Sample ordering is irrelevant to them.

3. For the only population the sample does address — stale ciphertext after a plaintext-only UPDATE — adding ORDER BY does not fix the stated scenario. A uniformly random 50-of-4000 sample still misses a single bad row ~98.8% of the time. The claim’s failure scenario survives its own proposed remedy, so the missing ORDER BY is not the cause of it. The genuine (and different) weakness is that a bounded spot check cannot prove whole-table ciphertext validity — which the file itself states at lines 23-24 and 56-58, calling it a “spot-check” over a “bounded sample” and never asserting coverage.
4. The “inflated count” sub-claim is wrong on the code’s own wording. `checked` sums per field pair (lines 97-101), so `savings_goals` contributes `50 target_amount_enc + 50 current_amount_enc` values. Line 115 says “ciphertext value(s) sampled” — 100 distinct ciphertext values is accurate. The claim’s arithmetic is also off: 8 field pairs across 7 JOBS is 400, not 350.
5. Reachability of bad ciphertext is thin. `processRow` (`encrypt-backfill.mjs`:137-174) re-SELECTs and decrypts every written row from the database before counting it, and `rollbackEnc` (196-211) NULLs any unverified write, returning it to the exact-count predicate. The only surviving path is a plaintext-only edit between backfill and 013; no route imports `lib/crypto.js` (`grep` shows only `node:crypto` for the cron HMAC at `server/routes/cron.js`:13), and the header (lines 20-21, 116) requires the gate and 013 run in one quiet window with the app paused or read-only.

Residual real element: without ordering, re-running the gate re-reads the same rows and adds no new confidence — a minor observability weakness, not a path by which migration 013 destroys data. Hence low, not high.

special_groups.name and recurrences.description are private free text in none of the three lists and absent from 012’s exclusion rationale

Lens: scope-coverage

Verified the literal observation but refuted the defect. The five-item block at `012_encryption_columns.sql`:11-22 does omit `special_groups.name` and `recurrences.description`, and `encrypt-backfill.mjs`:56-77 repeats the same omission. But: (a) `recurrences.description` is out of scope on substance, not oversight — `server/lib/runRecurrences.js`:15,63 copies it verbatim into `transactions.description` on every nightly cron insert, and `transactions.description` is deliberately plaintext because `routes/categories.js` `.like()`s it in the database (012:12-16); encrypting the recurrence copy would hide nothing since the identical string sits queryable in `transactions`. (b) The 012 block’s own heading is “Dropped from the original scope, deliberately” (012:11) — a delta against the file’s pre-2026-08-09 contents. 012 was originally written 2026-07-18; 014 and 015 are dated 2026-08-08, so neither column could ever have been in a list of things removed from that original scope. It is not an inventory. (c) The disposition is set by a stated rule, not an enumeration: 012:9 and `encrypt-backfill.mjs`:56-58 state “encrypt the MONEY, leave the searchable text alone”, and 012:21-22 gives the label rule (“Labels, not amounts... can be added later at low cost”). `special_groups.name` is a label; the rule assigns it plaintext unambiguously. `BUILD_PLAN.md`:732-735 records this as Alex’s explicit scope decision. (d) The claim that “no future reader can tell whether they were judged safe or simply missed” is factually false: `docs/2026-08-09-encryption-readiness-audit.md`:81-85 names both tables explicitly, including the exact “September 2026 Paris holiday” example, and :154 recommends extending scope to them; `BUILD_PLAN.md`:729-730 cites that audit as the source of the re-scope. The decision trail is on disk. (e) No mechanism is affected: the backfill and the 013

completeness gate iterate JOBS (encrypt-backfill.mjs:79-89, verify-encryption.mjs:34,96-110), not the comment block, so 013 can only drop plaintext columns that have `_enc` counterparts. No data loss, no gate failure, no broken feature — and the claimed “failure scenario” (those strings remain plaintext) is precisely the decided outcome of the 2026-08-09 money-only scope, which leaves ALL label text plaintext. Minor citation error in the claim: the Paris holiday example is at 015:3-4, not 015:12-13. Residual risk (someone later encrypting labels works from the five-item list and misses `special_groups.name`) is speculative future-authoring tidiness with no inputs-to-wrong-outcome path, and the audit doc already names the column — below this audit’s stated bar.

The on-disk migration 013 omits recurrences entirely, so the table 012 went out of its way to add is never actually encrypted

Lens: scope-coverage

Refuted on three independent grounds.

- (1) **FACTUAL ERROR IN THE HEADLINE.** There is no on-disk migration 013 — `ls server/migrations/013*` returns nothing, and `server/migrations/014_recurrences.sql:3-8` says so in prose (“013 is reserved ... and not yet written to disk”). The cited artefact is a planning document, not code and not a migration.
- (2) **FACTUAL ERROR IN THE CORE ASSERTION.** The claim says recurrences “is never actually encrypted.” It is. `server/migrations/012_encryption_columns.sql:52-54` adds `recurrences.amount_enc`; `server/scripts/encrypt-backfill.mjs:88` carries { `table: 'recurrences', fields: [['amount', 'amount_enc']]` } in JOBS; `server/scripts/verify-encryption.mjs:34,88,96-99` imports that same JOBS array and runs both `countUnencrypted` and `sampleDecrypts` over it. Backfill and gate both cover recurrences today. The only thing the plan block omits is the DROP of the plaintext column — a different and much narrower statement than the one made.
- (3) **THE FAILURE SCENARIO IS UNREACHABLE.** The plan’s SQL block (`docs/superpowers/plans/2026-07-17-phase9-pln-privacy-history-pace-special.md:574-599`) predates the 2026-08-09 re-scope and references five `_enc` columns that migration 012 does not create: `transactions.description_enc` (plan:577), `categories.name_enc` (plan:580), `savings_goals.name_enc` (plan:583), `savings_contributions.note_enc` (plan:588), `subscription_overrides.display_name_enc` (plan:590). Migration 012:32-54 adds only `transactions.amount_enc`, `budgets.amount_limit_enc`, `savings_goals.target_amount_enc/current_amount_enc`, `savings_contributions.amount_enc`, `user_stats.monthly_limit_enc`, `ask_messages.content_enc`, `recurrences.amount_enc` — the re-scope rationale is written out at 012:11-22. Pasting the plan block into the Supabase SQL editor therefore dies at plan:577 with column “`description_enc`” does not exist; a pasted multi-statement script runs inside an implicit transaction, so the entire block rolls back and NOTHING is dropped. Under the non-transactional reading it still halts loudly two statements in. Either way Alex is never “told the encryption work completed” — the described end-state (transactions/budgets/savings/user_stats/ask_messages stripped of plaintext while recurrences quietly retains it) cannot occur.

Additionally, Step 8 is explicitly gated on “Step 7 passes and Alex confirms” (plan:572), and Step 7 requires the route sweep of Step 6. No file under `server/routes` or `server/lib` imports `lib/crypto.js` (grep returns zero hits), so Step 6 has not begun. Reaching Step 8 requires a build session that must read migration 012 to know which columns exist, at which point the stale list is re-derived rather than pasted.

WHAT SURVIVES, AND IT IS NOT THIS CLAIM. The plan’s 013 block is genuinely stale and hazardous, but the recurrences omission is the mildest defect in it and the one already compensated for. The severe reading of the same block is `plan:579 alter table public.categories drop column name;` and `plan:576 drop column description` — both destroy columns the re-scope deliberately keeps in PLAINTEXT with no ciphertext counterpart anywhere, and `routes/categories.js` runs `.ilike()` on `transactions.description` (012:12-13). Whoever files the real finding here should file it against the whole stale block, not against recurrences.

Corrected severity: low. The observation points at a real doc-staleness hazard, but as stated the mechanism is wrong (recurrences IS encrypted and IS gated), the artefact does not exist on disk, and the failure path cannot execute. It is a stale planning document that will error out rather than a defect that loses data.

UNVERIFIED leads — no skeptic ever checked these

The refutation agents for the *operational* and *test-quality* lenses died on a session limit. These 12 findings are therefore **claims, not confirmed defects** — exactly the status the August audit’s lost 18 had. They are recorded here so they cannot be lost a second time. Given that 38% of the findings that *were* checked got refuted, expect a meaningful fraction of these to be noise.

[UNVERIFIED / claimed critical] Backfill encrypts a stale in-memory snapshot; a concurrent edit is silently overwritten and neither the row-verify nor the gate can see it

Where: `server/scripts/encrypt-backfill.mjs:110` (also `:132-134`, `:144-146`, `:154`)

Mechanism (unchecked). `keysetScan` fetches a page of up to 500 rows (`PAGE_SIZE=500`, :52) and holds them in memory. `processRow` encrypts from that snapshot (`encryptField(row.user_id, String(row[plain]))`, :110) and writes with an UNCONDITIONAL update — `update(patch)` filtered only by primary key (:132-134). There is no optimistic guard (no `.eq(plain, row[plain])`, no `updated_at` check). The ‘verified against the DATABASE’ step then re-SELECTs ONLY the `_enc` columns (`select(job.fields.map(([, e]) => e)...)`, :144) and compares them against `row[plain]` taken from the SAME in-memory snapshot (:154). The current database value of the plaintext column is never re-read, so a plaintext change that happened between the SELECT and the UPDATE is invisible to the check by construction. The header’s stated mitigation (:39-41, ‘a row inserted behind the cursor mid-run can be missed; a re-run is cheap, idempotent’) addresses INSERTs only; a re-run cannot repair this row because the idempotency filter `.is(firstEnc, null)` (:231) now excludes it — `amount_enc` is non-NULL.

Claimed failure scenario (unchecked). Backfill runs against `transactions` and reads a page containing tx X with `amount=50.00`. Before the UPDATE fires, the live app serves a PATCH correcting the amount to 75.00. The backfill writes `amount_enc = E(“50”)`; its re-read decrypts to “50”, compares against its own snapshot “50”, and reports success. Final row: `amount=75.00`, `amount_enc=E(“50”)`. `verify-encryption.mjs`’s only completeness query is plaintext-NOT-NULL AND enc-IS-NULL (`verify-encryption.mjs:45-53`) — both columns are non-NULL, so it counts the row as ‘ok’ and exits 0. Migration 013 drops `amount`. The user’s transaction is permanently 50.00 with no error logged anywhere, and the correct value no longer exists in any column.

[UNVERIFIED / claimed critical] Migration 013’s DROP+RENAME has no safe deploy ordering, and the 03:00 cron writes bare plaintext into the ciphertext column

Where: server/migrations/012_encryption_columns.sql:32-54 vs server/migrations/001_init.sql:59; server/vercel.json:11

Mechanism (unchecked). 013 will drop amount numeric(14,2) not null check (amount > 0) (001_init.sql:59) and rename amount_enc text (012:32-33, nullable, no CHECK) into its place. Three consequences. (1) The NOT NULL, the > 0 CHECK and the default 0 on savings_goals.current_amount (001_init.sql:93) all belonged to the dropped column and vanish — nothing then rejects a non-envelope value, because 012 adds no ~ '^v1:' constraint. (2) Old/rolled-back code writing a JS number into what is now text does not error; PostgREST coerces it and stores the bare digits. Vercel rollback to a previous deployment is one click, and Vercel’s per-request function instances are not swapped atomically with a SQL statement pasted into the Supabase editor. (3) The dual-write routes deployed immediately BEFORE 013 reference amount_enc, which after the rename does not exist — so 013 requires a THIRD deploy that the runbook (BUILD_PLAN.md:752-762: ‘...click-through on encrypted data, and ONLY THEN ... migration 013’) does not contain.

Claimed failure scenario (unchecked). Vercel Cron fires /api/cron/recurrences at 03:00 daily (vercel.json:11) — the same quiet window verify-encryption.mjs:21 tells the operator to use, and Vercel offers no maintenance mode to pause it. runRecurrences.js:58-67 inserts { amount: row.amount } as a number and its SELECT list (:14-15) has no amount_enc. Run 013 at 03:00: the sweep inserts transaction rows whose amount column holds the string “1200” instead of a v1 envelope. Every later read calls decryptField, which throws Unknown ciphertext version: 1200 (crypto.js:72); the global handler returns 500, so one poisoned row 500s the entire transactions list, dashboard and analytics for that user. The same coercion happens through goals.js:175 .update({ current_amount: newAmount }) on any un-swept or rolled-back instance, and goals.js:42/164/165 Number(goal.current_amount) yields NaN once the column holds ciphertext.

[UNVERIFIED / claimed critical] Nothing proves the backfill’s key and production’s key are the same, and the envelope carries no key id — so a key mismatch is undetectable until after the plaintext is dropped

Where: server/lib/crypto.js:38-56 (and server/.env.example, which has no DATA_ENCRYPTION_KEY entry)

Mechanism (unchecked). masterKey() validates the key’s SHAPE only — canonical base64, 32 bytes, round-trip (:38-51). Any other valid 32-byte base64 string passes every check. The stored envelope is v1:<iv>:<tag>:<ct> (:64) where ‘v1’ is the FORMAT version; there is no key id and no key check value, and userKey() derives from a fixed salt and user:\${userId} info only (:56). The backfill runs on Alex’s laptop against server/.env; the deployed app reads Vercel’s env var; verify-encryption.mjs also runs locally against server/.env (:132-140). No component ever compares the two. server/.env.example — the committed template SECURITY.md:110 designates as the record of required vars — does not list DATA_ENCRYPTION_KEY at all, so a mistyped or re-generated Vercel value has nothing to be checked against.

Claimed failure scenario (unchecked). Alex generates key A, backfills locally with it, then pastes a re-generated key B into Vercel (or truncates/re-generates during a later env edit). During the dual-write phase the routes still READ plaintext — that is what makes dual-write reversible — so key B produces no error at all: rows written by production simply carry ciphertext only

key B can open. The local gate runs under key A, and its sample of 50 unordered rows (verify-encryption.mjs:60-66) returns heap-order rows, i.e. the backfilled ones, all of which decrypt. It prints PASS. 013 drops the plaintext. Every row written between the deploy and 013 is now permanently unreadable, and because the envelope has no key id there is no way to tell those rows from genuinely corrupt ones, nor to run a dual-key repair. The same absence means DATA_ENCRYPTION_KEY can never be rotated after 013 — a key leaked into a build log has no remedy, unlike the service-role key SECURITY.md:136 assumes is rotatable.

[UNVERIFIED / claimed critical] The migration-013 gate has zero tests and its only garbage-detector samples 50 unordered rows — it returns PASS on a database containing undecryptable ciphertext

Where: server/scripts/verify-encryption.mjs:60-67, :98-112

Mechanism (unchecked). verifyEncryption is exported with an injectable supabase/jobs/sample/log — exactly the shape runBackfill uses to be testable — yet grep over routes/lib/scripts/test finds NO reference to verifyEncryption outside its own file. It is the single authority for an irreversible DROP and it has never been executed against any fixture. Worse, its two checks have complementary blind spots. countUnencrypted (line 45) only counts plaintext NOT NULL AND enc IS NULL, so a row whose _enc is non-NULL but garbage (the exact residue a failed rollback leaves, per encrypt-backfill.mjs:179-195) contributes 0. The only thing that could catch it is sampleDecrypts, which issues .not(enc,'is',null).limit(limit) with NO .order() and no randomisation, default DEFAULT_SAMPLE = 50 (line 36). Postgres with no ORDER BY returns physical order, so the ‘sample’ is the same first 50 rows on every run, forever.

Claimed failure scenario (unchecked). I built a PostgREST-faithful fake (honours .not/.is/head-count/.limit, physical row order) with 600 transactions across 5 users, and set row #400’s amount_enc to a value encrypted under the wrong user id — a stranded/mis-keyed row. verifyEncryption(...) with the default sample returned {"pass":true,"missing":0,"checked":50,"failures":[]}. The same database with sample: 600 returned pass:false, failures:["transactions id=0400 amount_enc: will not decrypt"]. So the gate prints ‘PASS — nothing would be lost’ and ‘Migration 013 may proceed’, 013 drops transactions.amount, and that row’s amount is permanently unrecoverable. On a 5-user live table any corruption past physical row 50 is invisible to the gate by construction.

[UNVERIFIED / claimed critical] Three of the eight encrypted column pairs can be deleted from JOBS with the whole suite green — and verify-encryption imports the same JOBS, so the gate stops checking them too

Where: server/test/encrypt-backfill.test.js:181-189 (guarding server/scripts/encrypt-backfill.mjs:78-89 and server/scripts/verify-encryption.mjs:34)

Mechanism (unchecked). JOBS is the single source of truth for both the writer (encrypt-backfill) and the gate (verify-encryption.mjs:34 import { JOBS } from './encrypt-backfill.mjs'), so a column absent from JOBS is never encrypted AND never checked — the gate reports PASS by simply not looking. The only test pinning JOBS asserts 4 of the 8 pairs present (transactions.amount, budgets.amount_limit, user_stats.monthly_limit, recurrences.amount) and 2 absent. savings_goals.current_amount, savings_contributions.amount and ask_messages.content are asserted nowhere, and nothing cross-checks JOBS against the 8 _enc columns migration 012 actually adds (012_encryption_columns.sql:32-54).

Claimed failure scenario (unchecked). I deleted the `ask_messages` job, the `savings_contributions` job, and the `current_amount` pair from `savings_goals` — leaving 5 of 8 pairs — and ran the suite: 28 pass, 0 fail. In production that means the backfill never writes `ask_messages.content_enc`, `savings_contributions.amount_enc` or `savings_goals.current_amount_enc`; `verify-encryption` never iterates those pairs so `missing` stays 0 and it prints PASS; migration 013 (written by hand from migration 012’s column list, not from JOBS) drops the plaintext `content`, `amount` and `current_amount` columns. Every Ask Trim transcript and every savings contribution for all 5 users is destroyed with no ciphertext to recover from.

[UNVERIFIED / claimed high] The completeness gate authorising the irreversible drop cannot detect stale ciphertext, samples with no ORDER BY, and has zero tests

Where: `server/scripts/verify-encryption.mjs:45-53` and `:60-66`

Mechanism (unchecked). `countUnencrypted` asks exactly one question: `.not(plain,'is',null).is(enc,null)` — plaintext present AND ciphertext absent. A row where BOTH columns are non-NULL but the ciphertext is older than the plaintext is counted as ‘ok’. The decrypt spot-check is the only thing that could catch it, and it is `.limit(limit)` with NO `.order()` (`:63-66`), default sample 50 (`:36`). PostgreSQL without an ORDER BY returns rows in unspecified/heap order, so the sample is systematically the oldest rows — precisely the ones the backfill wrote correctly — and systematically excludes rows written after it. There is also no assertion that `checked > 0`: the PASS branch prints ‘nothing would be lost’ for a sample of zero (`:114-115`). The gate’s scope is JOBS imported from the backfill (`:34`), so it can only check columns someone remembered to list; migration 013 is hand-written and nothing ties its DROP list to JOBS — and the scope was already re-decided once (`012:5-22`), removing five columns from the original list.

Claimed failure scenario (unchecked). Any write path not converted by the route sweep updates a plaintext column after the backfill. Concretely, `goals.js:175` today does `.update({ current_amount: newAmount })` on every savings contribution and touches no `_enc` column: a goal backfilled at `current_amount=200` receives a £50 contribution, ending at `current_amount=250`, `current_amount_enc=E(“200”)`. Both non-NULL, so `missing` stays 0; the 50-row unordered sample does not reach it. The gate exits 0, 013 drops the plaintext, and the goal permanently shows £200 — the contribution row still exists, so the goal no longer reconciles with its own contribution history. Because the script has no test file (`server/test/` contains `crypto.test.js` and `encrypt-backfill.test.js` but nothing for `verify-encryption`), none of this is exercised anywhere before it is trusted with an irreversible DROP.

[UNVERIFIED / claimed high] transactions.original_amount and fx_rate are money left in plaintext beside the encrypted amount, reconstructing it exactly

Where: `server/migrations/016_foreign_currency.sql:21-31` (absent from 012 and from `encrypt-backfill.mjs:78-89`)

Mechanism (unchecked). Migration 016 (2026-08-10, applied to live) added `original_amount` `numeric(14,2)` and `fx_rate` `numeric(18,8)` to `transactions`. Its own header states the conversion rule: `amount` is `original_amount` converted at entry (`016:5-8`). Migration 012 was re-scoped on 2026-08-09 — one day earlier — and covers only `transactions.amount` (`012:32-33`); JOBS lists only `['amount', 'amount_enc']` for that table (`encrypt-backfill.mjs:79`). Because the gate derives its scope from JOBS (`verify-encryption.mjs:34`), it cannot flag a money column nobody listed. This is the identical failure mode the 2026-08-09 audit found for `recurrences` and `special_groups` (`docs/2026-`

08-09-encryption-readiness-audit.md:73-90), recurring because nothing ties the encrypted-column list to the schema.

Claimed failure scenario (unchecked). Alex logs €45.00 while travelling; the row stores `amount=38.50` (encrypted post-013), `original_amount=45.00`, `fx_rate=0.85565` — the latter two in cleartext, in the same row. Anyone glancing at the Supabase table editor — the exact threat the feature exists to stop (audit doc:130-131) — multiplies 45.00×0.85565 and recovers 38.50 for every foreign-currency transaction. The rollout completes, the gate exits 0, migration 013 runs, and Alex is told his amounts are encrypted while his travel spending is fully readable. `dashboard.js:39` already selects all three columns together, so the pairing is present in normal API responses too.

[UNVERIFIED / claimed high] The one unrecoverable secret is absent from every operational document and every startup check, and no backup precedes the irreversible drop

Where: `SECURITY.md:129-141` (deployment checklist) and `server/index.js:23-27`

Mechanism (unchecked). `crypto.js:6` tells the operator ‘Losing `DATA_ENCRYPTION_KEY` = losing every user’s data. See `SECURITY.md`.’ — a dangling reference: `grep` for `encrypt|DATA_ENCRYPTION_KEY|at rest` across `SECURITY.md` returns zero hits. It is not in the key table (`SECURITY.md:11-19`), not in the ‘Deployment checklist (before any deploy)’ (`:129-141`) that lists `CLIENT_URL`, `ANTHROPIC_API_KEY` and `CRON_SECRET`, and not in `server/.env.example`. `index.js:23-27` fails fast on `CLIENT_URL` only, despite `SECURITY.md:111` mandating ‘Add similar fatal checks for any new required env var’ — so a missing key surfaces per-request inside `masterKey()` (`crypto.js:33`) as a generic 500, not at boot. Separately, no runbook contains a backup step: `DEPLOY.md:56-59` documents the project as free-tier Supabase (‘upgrade the project to Pro (\$10/mo), or accept that you must Restore it’), which has no PITR and no restorable scheduled backups, and `BUILD_PLAN.md:45` records the project having already been paused and restored once. The audit’s own recommended sequence (`docs/2026-08-09-encryption-readiness-audit.md:163`) is ‘key -> 012 -> dry run -> backfill -> sweep -> click-test -> gate -> 013’ — no snapshot anywhere.

Claimed failure scenario (unchecked). Alex works through `SECURITY.md`’s deployment checklist before the encryption deploy — the only checklist that exists — and it never mentions the key, so `DATA_ENCRYPTION_KEY` is set locally for the backfill but omitted from Vercel. The server boots healthy (`/api/health` returns ok) and only starts throwing when a user submits a transaction, appearing as an intermittent app fault rather than a config error. If the key is instead lost or overwritten at any point after 013 — laptop failure, an env edit, a Vercel project re-link — there is no plaintext column, no key backup mandated by any committed document, and no database snapshot or PITR on the current tier to restore from: five people’s complete financial history is permanently unreadable.

[UNVERIFIED / claimed high] parseArgs test overclaims: a dashless typo such as `node encrypt-backfill.mjs dry-run` performs a full live write pass, and no test covers that class

Where: `server/scripts/encrypt-backfill.mjs:328` (test at `server/test/encrypt-backfill.test.js:173-179`)

Mechanism (unchecked). `parseArgs` filters `argv.filter((a) => a.startsWith('-'))`, so only dash-prefixed tokens are ever inspected. Anything without a leading dash falls through the `unknown-flag`

guard entirely and `argv.includes('--dry-run')` is false, yielding `{dryRun:false}` — a live run. The test is named ‘parseArgs refuses an unknown flag rather than performing a live run’ but enumerates only `--dryrun`, `--dry_run`, `--dry-run=true`, `-d`; every one of those has a dash, so the test certifies exactly the half of the input space the guard actually covers and none of the half it does not.

Claimed failure scenario (unchecked). Verified directly against the module: `parseArgs(['dry-run']) → {dryRun:false}`, `parseArgs(['dryrun']) → {dryRun:false}`, `parseArgs(['dry_run']) → {dryRun:false}`, `parseArgs(['DRY-RUN']) → {dryRun:false}`. Only `-dry-run` throws. So Alex, intending a preview against the live database, types `node scripts/encrypt-backfill.mjs dry-run`, sees no error, and the script writes encrypted values into every `_enc` column across 7 tables for 5 real users — the script’s own header (lines 43-46) warns this must not happen before `DATA_ENCRYPTION_KEY` is generated and backed up. If the key in `.env` is not yet backed up, that write pass is unrecoverable ciphertext.

[UNVERIFIED / claimed high] The one unrecoverable state — rollback fails or silently does not land — has no test at all; the fake never injects an update error and the rollback write is never re-read

Where: `server/scripts/encrypt-backfill.mjs:196-211` and `:134-135` (fake at `server/test/encrypt-backfill.test.js:20-74`)

Mechanism (unchecked). `makeFake`’s `then()` for `_mode === 'update'` unconditionally resolves `{error: null}` (line 61) — there is no `faults.updateError` and no `faults.rollbackError`. So if `(upErr) throw upErr` (line 135) is untested, and the entire `catch` block in `rollbackEnc` (lines 204-210) — the block that appends the ‘!! ROLLBACK ALSO FAILED ... do NOT run migration 013’ warning, described in the doc comment as ‘the one state a re-run cannot repair on its own’ — is never entered by any test. The rollback also never re-reads to confirm the NULL landed, even though step 3 exists precisely because a write can ‘silently not land’. The fake is generally low-fidelity here: `not()` and `order()` are no-op passthroughs (lines 33-34) and `select()` ignores its `options` argument entirely, so PostgREST filters and `{count:'exact', head:true}` are unmodelled.

Claimed failure scenario (unchecked). I replaced the whole `catch { cause.message += ... }` body in `rollbackEnc` with an empty comment: 28 pass, 0 fail. In production: PostgREST returns a statement-timeout on the step-3 re-read for transaction id X; the code calls `rollbackEnc`, whose `UPDATE` also fails (same overloaded connection) or returns `{error:null}` without landing. Because the warning branch is unverified, Alex sees only `VERIFY FAILED` (row not found on re-read after write) transactions `id=X`. He re-runs the backfill as the header invites; `keysetScan`’s `.is(amount_enc, null)` filter (line 231) no longer matches row X, so it is silently skipped; ‘Backfill complete’ prints. `verify-encryption`’s `countUnencrypted` also skips it (`enc` is non-NULL) and its 50-row sample misses it. 013 drops `transactions.amount` and row X’s amount is unrecoverable ciphertext.

[UNVERIFIED / claimed medium] The `auth-tag` test asserts on the message of the redundant length check, so the `authTagLength` option can be deleted from both cipher and decipher with 28/28 green

Where: `server/test/crypto.test.js:110-116` (guarding `server/lib/crypto.js:62` and `:90`)

Mechanism (unchecked). `crypto.js:82-84` states the design as belt-and-braces: `authTagLength: TAG_BYTES` makes Node enforce the 16-byte tag, and the explicit `if (tag.length !== TAG_BYTES)`

check at :86-88 is the documented survivor of ‘a future refactor that drops the option’. The only test asserts `/auth tag must be 16 bytes, got 4/` — the literal message of the explicit check. Pinning the message means the test can only ever observe the explicit check firing; it is structurally incapable of observing whether the `authTagLength` option is present. The intended relationship is therefore backwards: the redundant guard is tested, the primitive-level guard is not.

Claimed failure scenario (unchecked). I removed `, { authTagLength: TAG_BYTES }` from both `createCipheriv` (line 62) and `createDecipheriv` (line 90): 28 pass, 0 fail — a completely silent regression leaving the $2^{128 \rightarrow 232}$ forgery protection resting on one hand-written `if`. I then also removed lines 86-88 (the refactor the code comment explicitly anticipates) and confirmed the consequence empirically: `decryptField('u1', <envelope with tag truncated to 4 bytes>)` returned `'secret'`, i.e. Node accepted a 4-byte GCM tag with only a deprecation warning, and only 1 of 19 crypto tests failed. A write-capable attacker (leaked service-role key — the threat this module names) forges any amount at 2^{32} work instead of 2^{128} .

[UNVERIFIED / claimed medium] `offsetScan` is unreachable dead code with zero tests, so `alreadyEncrypted` is permanently 0 and a re-run’s output cannot be distinguished from the script finding nothing at all

Where: `server/scripts/encrypt-backfill.mjs:257-282, :292-300` (test at `server/test/encrypt-backfill.test.js:154-163`)

Mechanism (unchecked). `runJob` picks `offsetScan` only when `pkOf(job).length !== 1`. Every entry in `JOBS` has a single-column PK — including `user_stats`, whose `pk: ['user_id']` has length 1 — and `subscription_overrides`, the composite-PK table the function’s own doc comment (line 250) names, is not in `JOBS` at all. `counts.alreadyEncrypted += 1` at line 273 is the sole increment site and lives inside `offsetScan`, so on every reachable path the counter stays 0 and the `already encrypted` clause at line 298 can never print. The test ‘already-encrypted rows are skipped, so a re-run is genuinely idempotent’ therefore asserts only `second.encrypted === 0` — the same observable as ‘scanned nothing’.

Claimed failure scenario (unchecked). I replaced `offsetScan`’s body with `throw new Error('offsetScan is UNREACHABLE')`: 28 pass, 0 fail, confirming zero coverage and zero reachability. The operator-facing consequence, reproduced against a fake: a `transactions` table where 3 of 4 rows are already encrypted logs `transactions: 1 rows encrypted + verified against the database`; a table that is completely empty logs `transactions: 0 rows encrypted + verified against the database`. After a fully successful first run, the confirming re-run prints `0 rows encrypted` with no mention that 5,000 rows were skipped as already-done — byte-identical to what a typo’d table name, a wrong Supabase project, or an empty table would print. That output is the operator’s evidence for proceeding to the irreversible migration 013, and it carries no information.